

Server-Factory-Additional-Components

La distribution secondaire de la chaîne d'outillage de provisionnement Server Factory.

Résumé

Au-delà de Mail Server Factory et du Cadre de base, l'organisation Server-Factory comprend plusieurs composants plus modestes : des « usines » par service (Service Web, SonarQube, Proxy de mise en cache), des packs de configuration déclarative (Docker/Stack/Définitions logicielles) et des Utilitaires partagés. Cette page les présente sans fard — plusieurs sont en phase préliminaire ou à peine documentés — plutôt que comme des produits pleinement spécifiés.

Description succincte

Un ensemble de dépôts d'accompagnement pour Server Factory : Web-Service-Factory, SonarQube-Factory et Caching-Proxy-Factory (outils de provisionnement par service, majoritairement en phase initiale) ; Docker/Stack/Software-Definitions (packs de configuration déclarative utilisés par le cadre) ; et Utils (aides à l'accès SSH et outils généraux). Tous s'appuient sur le Cadre de base.

Description détaillée

Cette page regroupe les dépôts restants de Server-Factory car, pris individuellement, la plupart sont de taille réduite ou volontairement sous-documentés, et les présenter comme des produits finis enjoliverait leur degré de maturité. Ils se répartissent en trois catégories. **Les usines de services** suivent le modèle Mail Server Factory pour d'autres rôles serveur : **Caching-Proxy-Factory** (« Exécutez votre propre serveur proxy de mise en cache ») met en avant comme fonctionnalités clés un proxy de mise en cache, un certificat auto-signé et un point de

terminaison HTTP pour l'obtention de certificats de sécurité ; **SonarQube-Factory** (« Exécutez votre propre serveur SonarQube ») cible les usages en développement logiciel ; et **Web-Service-Factory** instancie et configure un serveur web pour déployer des cibles telles que des sites web et des micro-services. Tous trois sont des projets Kotlin bâtis sur le Cadre de base, mais leurs fichiers README publics restent largement des ébauches (« À définir » pour la compatibilité, les spécifications, la configuration et l'utilisation) — leurs capacités concrètes, au-delà de l'intention déclarée, sont donc NON VÉRIFIÉES. **Les packs de définitions — Docker-Definitions, Stack-Definitions et Software-Definitions** — sont des dépôts de configuration déclarative que le cadre utilise pour savoir comment construire/déployer des images Docker, des piles logicielles et des logiciels ; il s'agit de packs de données versionnés plutôt que d'applications. **Utils** fournit des outils généraux pour l'ensemble de la famille, dont un script `init_ssh_access.sh` générant une clé SSH et l'installant sur un hôte distant pour permettre un accès root sans mot de passe lors des provisionnements ultérieurs. Ensemble, ces composants complètent la chaîne d'outillage de provisionnement autour du produit phare Mail Server Factory.

Pourquoi nous l'avons conçu

Le modèle Server Factory est conçu pour être généralisable : une fois que l'on peut provisionner un serveur de messagerie à partir d'une description déclarative, le même moteur devrait pouvoir provisionner des serveurs web, des proxys de mise en cache et des serveurs d'analyse de code — alimentés par des packs de définitions réutilisables et des utilitaires partagés plutôt que par une logique sur mesure pour chaque rôle. Ces dépôts incarnent cette généralisation en cours, étendant le modèle éprouvé à de nouveaux types de serveurs. Leur valeur réside ici dans la démonstration de la portée du modèle ; leur maturité varie, et cette page précise délibérément ce qui relève de la vision et ce qui est abouti.

Contenu

Pourquoi c'est un changement majeur (mesuré)

Ensemble, ces éléments démontrent la réutilisabilité du Cadre de Base (*Core Framework*) sur différents types de serveurs et séparent clairement les données déclaratives (Définitions) de l'exécution (usines). Individuellement, les usines de services en sont à un stade précoce et doivent être présentées comme une orientation, et non comme des produits finis.

Ce qui est innovant

- Un cadre de provisionnement généralisé pour les rôles de messagerie, web, proxy-cache et SonarQube.
- Des packs de Définitions déclaratives (Docker/Stack/Logiciel) découplés du moteur d'exécution.
- Des Utilitaires partagés (ex. : un bootstrap sans mot de passe en une commande via SSH) réutilisés dans toutes les usines.

Défis et solutions

- **Réutiliser un même moteur pour différents rôles de serveur** : résolu en construisant chaque usine sur le Cadre de Base.
- **Séparer la configuration du code** : résolu grâce aux dépôts de Définitions, packs de données versionnés.
- **(NON VÉRIFIÉ)** : les fichiers README des usines de services sont des placeholders ; leur niveau d'implémentation ne peut être vérifié à partir des documents publics — à présenter comme un stade précoce.

Pile technologique (pourquoi + comment)

- **Kotlin** — Usine de services web, Usine SonarQube, Usine de proxy-cache (construites sur le Cadre de Base).
- **Shell** — Utilitaires et packs de Définitions (scripts/config).
- **Gradle** — Flux de construction/test `./gradlew test` pour l'ensemble des usines.
- **Docker** — Environnement d'exécution cible décrit par les Définitions Docker.
- **SSH / OpenSSH** — Bootstrap des Utilitaires pour un accès sans mot de passe.
- **SonarQube** — Le serveur provisionné par l'Usine SonarQube (et pour lequel Mail Server Factory signale une validation sans faille).

Note de transparence : la plupart de ces dépôts sont des forks au sein de l'organisation ; les usines de services sont documentées comme des placeholders et marquées NON VÉRIFIÉES conformément à l'article §11.4.6 de la constitution. Leur importance est clairement inférieure à celle de Mail Server Factory et du Cadre de Base.